# CyberSecIntel
# P2 First Deliverable
# Trusted Zone, Exploitation Zone, and Data Consumption: Architecture Design

Dídac Cayuela and Sindri Masson
Big Data Management – Universitat Politecnica de Catalunya

April 2026

## 1 Architecture Design

### 1.1 Design approach

The P2 architecture follows Option 1 from the project statement: the existing MinIO and Delta Lake stack is extended with new buckets and table joins rather than replaced with new database engines. The reasoning is practical and grounded in the actual data characteristics.

The P1 sources produce at most a few hundred thousand CVE records, a few million Suricata events per replay run, and a modest IOC feed. Delta Lake on MinIO handles all of these comfortably through Spark DataFrame operations. Introducing a columnar OLAP engine or a document store at this scale and with this query complexity would not improve query latency or developer productivity in a way that justifies the added operational overhead. A single storage technology is used throughout all three pipeline zones: MinIO with Delta Lake tables.

The end-to-end pipeline therefore extends the P1 framework as follows:

1. **Sources** → **Landing Zone** (`s3://landing/`, Delta): unchanged from P1.
2. **Landing Zone** → **Trusted Zone** (`s3://trusted/`, Delta): Spark ETL; information-preserving cleaning.
3. **Trusted Zone** → **Exploitation Zone** (`s3://exploitation/`, Delta): Spark ETL; join, enrich, correlate, and score.
4. **Exploitation Zone** → **Consumption**: five parallel outputs serving distinct SOC analyst workflows.

*[Architecture diagram – place `assets/BDM_P2.drawio.png` here]*

Figure 1: CyberSecIntel updated architecture design (P2). Storage remains MinIO + Delta Lake throughout all three pipeline zones. Spark is the sole transformation engine across batch, streaming, and ML paths. The five consumption endpoints cover cold, warm, and hot data paths.

### 1.2 Technology summary

| Technology | Zone(s) | Role and justification |
|---|---|---|
| Apache Spark (PySpark) | Trusted, Exploitation, Consumption | The single transformation engine across all zones. Handles batch ETL (cleaning, joining, correlating, aggregating) for the cold path, Structured Streaming for the warm path, and ML training and inference for the anomaly feed. The Spark SQL Thrift Server exposes Delta tables to Grafana via a persistent SQL endpoint, removing the need for a separate query engine. |
| MinIO + Delta Lake | Landing, Trusted, Exploitation | Same stack as P1, extended to three zones. Delta provides ACID commits, schema enforcement, versioned history, and native Spark integration. All pipeline data, including KPI aggregates, warm enriched alerts, and anomaly flags, remains in MinIO as Delta tables. |
| Apache Kafka | Landing, Consumption | Unchanged from P1. The `ids.alerts` topic feeds the warm-path Spark Structured Streaming job; `suricata.events` is surfaced directly as the live alert stream. |
| Grafana | Consumption | Dashboarding service that queries exploitation zone Delta tables via the Spark SQL Thrift Server. The only new Docker service introduced in P2. |

### 1.3 Cold, warm, and hot paths

The **cold path** is extended through two new Airflow DAGs running sequentially after the P1 landing DAGs. Each DAG uses Spark to read from the previous zone's Delta tables and write cleaned or enriched tables to the next zone's bucket in MinIO.

The **warm path** originates at the Kafka `ids.alerts` topic published by the P1 Suricata replay pipeline. A Spark Structured Streaming job reads from this topic in micro-batch mode, enriches each alert against the `exploitation/vuln\_enriched` Delta table loaded as a static broadcast, and writes results to the warm enriched Delta table in the exploitation zone. The warm aggregates accumulated in the landing zone are also processed by the trusted zone batch ETL, providing a governed historical record alongside the real-time stream.

The **hot path** (`suricata.events`) remains pass-through: raw EVE events are time-critical and already normalized at publish time by the P1 replay pipeline. They are surfaced directly as the live alert stream without further processing.

### 1.4 Streaming design

P1 established a Kafka-backed hot path that publishes Suricata EVE events and IDS alerts to two topics. P2 extends this with a warm-path processing step on `ids.alerts`.

**Hot path (`suricata.events`).** Raw EVE events are time-critical and already normalized at publish time. Applying further transformations would add latency without benefit. The hot path therefore remains pass-through, surfaced as the live alert stream.

**Warm path (`ids.alerts`).** IDS alerts have lower volume and tolerate slightly higher latency: enriching an alert with a vulnerability priority score within seconds is sufficient for SOC triage. Spark Structured Streaming reads from the `ids.alerts` Kafka topic in micro-batch mode (10-second interval), joins each batch against `exploitation/vuln\_enriched` loaded as a static broadcast DataFrame, and writes enriched records to the warm enriched Delta table in `s3://exploitation/`.

**Tool choice.** Spark Structured Streaming is used rather than Kafka Streams or Apache Flink because Spark is already the transformation engine for the batch zones. Reusing it avoids introducing a second framework. The `spark-sql-kafka` connector provides native Kafka source support.

## 2 Trusted Zone

### 2.1 Design goals and principles

The trusted zone applies information-preserving transformations: operations that correct representational errors without encoding task-specific assumptions. Concretely, this means deduplication by natural key, type casting, date standardization, structural flattening of nested fields, and validation against expected value ranges or enumerations. It explicitly excludes outlier removal, missing-value imputation, and normalization steps that assume a particular downstream model.

The heuristic applied throughout is: *if two downstream tasks with different requirements consumed this asset, would both benefit from this transformation, or would one of them be harmed?* Only transformations that pass this test are admitted to the trusted zone.

### 2.2 Storage

Each zone holds two primary data threads and one zone-specific object. In the trusted zone: **Delta clean tables** (`s3://trusted/`) contain cleaned versions of each landing zone source, partitioned by source family; **warm cleaned Delta** contains the cleaned IDS alert aggregates flowing from

the landing zone warm aggregates; and **rejection tables** (`s3://trusted/rejected/`) collect rows excluded by mandatory-field or integrity checks, retaining them for audit without dropping them from the pipeline.

### 2.3 Cleaning transformations per data type

**Structured data (KEV, EPSS).**

- **Deduplication.** KEV on `cveID`; EPSS on compound key `(cve, date)`.
- **Type casting.** Date strings (`dateAdded`, `dueDate`) are cast to `DateType`. EPSS `epss` and `percentile` are cast to `DoubleType`.
- **Range validation.** EPSS scores outside $[0, 1]$ receive a Boolean `\_quality\_warn` flag and are retained rather than dropped, preserving the row for audit.
- **Null enforcement.** Rows with null mandatory fields (`cveID`, `vulnerabilityName` for KEV; `cve`, `date` for EPSS) are written to `trusted/rejected/` rather than silently discarded.

**Semi-structured data (NVD, URLhaus, CIRCL, ThreatFox).**

- **Schema alignment.** Field names are normalized to snake_case (`cveId` → `cve\_id`, `lastModified` → `last\_modified`).
- **Flattening.** NVD CVSS blocks are flattened: `metrics.cvssMetricV31[0].cvssData` fields are promoted to top-level columns (`cvss\_v3\_score`, `cvss\_v3\_vector`, `cvss\_v3\_severity`). Missing CVSS blocks produce null columns rather than dropped rows.
- **Missing field defaults.** Optional fields absent in some records receive typed nulls, ensuring a stable column set for downstream Spark jobs.
- **Deduplication.** Natural keys: `cve\_id` for NVD and CIRCL; `id` for URLhaus and ThreatFox.
- **URL validity.** URLhaus `url` values are validated against a basic pattern; invalid entries receive `\_quality\_warn=true`.

**Unstructured data (CTU-13 PCAPs, Suricata EVE JSONL).**

- **PCAP integrity.** Each artifact's SHA-256 is recomputed and compared against the sidecar `.meta.json` written at landing time. Mismatches set an `integrity\_fail` flag in the `pcap\_artifacts` Delta table; the file is not deleted.
- **EVE JSONL cleaning.** Suricata EVE records are validated for required fields (`event\_type`, `timestamp`, `src\_ip`). Malformed timestamps receive a sentinel value and a flag. Encoding is standardized to UTF-8. Duplicate events within the same replay run (identified by `flow\_id` + `timestamp` + `event\_type`) are dropped.

**Warm aggregates (IDS alert aggregates).** The warm aggregates Delta accumulated in the landing zone (alerts per minute, top source IPs, top signatures) are cleaned by the trusted zone batch ETL: deduplication by time window key, type validation on count fields, and null enforcement on the aggregation dimensions. The result is written to the warm cleaned Delta table, providing a governed historical record alongside the real-time warm path.

### 2.4 Orchestration

A new Airflow DAG `cybersecintel\_trusted\_zone` depends on completion of both P1 ingestion DAGs. It runs one Spark task per source family and one task for the warm aggregates, writing cleaned Delta tables to `s3://trusted/`.

# 3 Exploitation Zone

## 3.1 Design goals

The exploitation zone produces data ready for analyst consumption: pre-joined tables that eliminate repeated join work, IOC-correlated network events, a composite vulnerability priority score, anomaly-flagged flows, and pre-aggregated KPI tables. All data remains in Delta Lake on MinIO, queried by Grafana via the Spark SQL Thrift Server.

## 3.2 Storage

The exploitation zone holds two primary data threads and one zone-specific object. **Enriched Delta tables** (`s3://exploitation/`) contain the joined and correlated analytical assets produced by the cold batch path. **Warm enriched Delta** contains the continuously updated IDS alerts enriched with CVE context by the Spark Structured Streaming job. **KPI tables** (`s3://exploitation/kpi/`) hold pre-aggregated metrics recomputed by the exploitation DAG on each run and queried directly by Grafana.

## 3.3 Data organization and modeling

**`exploitation/vuln\_enriched` Delta table.** The central analytical asset is a wide, pre-joined CVE table produced by left-joining `trusted/kev`, `trusted/nvd`, `trusted/epss`, and `trusted/circl\_vulnlookup` on `cve\_id`. The result is one row per CVE containing KEV exploit status and dates, NVD CVSS scores and description, EPSS probability and percentile, CIRCL CWE and reference counts, and a computed `vuln\_priority\_score` column.

**`exploitation/network\_events` Delta table.** A union of `trusted/suricata\_events` and `trusted/ids\_alerts` produces a unified network security event log with a consistent schema: event timestamp, type, source and destination IP and port, protocol, alert signature, severity, category, scenario, and origin source. The table is partitioned by ingestion date for efficient time-range scans.

**`exploitation/ioc\_correlations` Delta table.** A Spark job joins the `trusted/threatfox` and `trusted/urlhaus` IOC tables against `exploitation/network\_events` on source and destination IP addresses and observed URLs. Each matching record carries the IOC type (IP, domain, or URL), the associated malware family from ThreatFox, and the original network event context. This table feeds the IOC correlation feed in the consumption layer.

**`exploitation/anomaly\_flags` Delta table.** A Spark batch job reads `exploitation/network\_events` and extracts a feature matrix (alert category, severity, source port, destination port, event count per flow). A trained scikit-learn Isolation Forest model scores each flow; records below the contamination threshold are written to this table with their anomaly score and feature values. The trained model artifact is persisted to `s3://models/ids\_anomaly\_detector/` for versioning and reuse.

**KPI tables (`s3://exploitation/kpi/`).** Three pre-aggregated Delta tables are recomputed by the exploitation DAG and queried by Grafana:

- `kpi\_vuln\_priority\_top50`: the top 50 CVEs by `vuln\_priority\_score` with key fields for the CVE prioritization feed.
- `kpi\_daily\_alert\_counts`: alert count grouped by date, severity, and signature for the alert trend panel.
- `kpi\_kev\_rolling7d`: count of CVEs added to KEV in the last seven days for the headline stat panel.

### 3.4 Computed new assets

**Vulnerability priority score.**

$$priority = epss_score \times \underbrace{(1.5 \text{ if kev_listed else } 1.0)}_{\text{exploit confirmation bonus}} \times \frac{cvss_v3_score}{10}$$

This composite score encodes three complementary signals: EPSS provides the statistical probability of exploitation in the near term; the KEV bonus elevates CVEs confirmed as actively exploited in the wild; CVSS provides the technical severity weight.

### 3.5 Orchestration

A new Airflow DAG `cybersecintel\_exploitation\_zone` depends on `cybersecintel\_trusted\_zone`. It runs five tasks: the join task (produces `vuln\_enriched`), the union task (produces `network\_events`), the IOC correlation task (produces `ioc\_correlations`), the ML task (produces `anomaly\_flags` and updates `s3://models/`), and the KPI task (writes aggregate tables to `s3://exploitation/kpi/`). All tasks are idempotent: Delta writes use overwrite mode and KPI tables are fully recomputed on each run.

## 4 Data Consumption

The five consumption endpoints cover all three data paths and deliver distinct business value to SOC analyst workflows. Grafana serves the four cold and warm endpoints by querying exploitation zone Delta tables via the Spark SQL Thrift Server. The live alert stream bypasses Grafana entirely.

### 4.1 CVE prioritization feed

Grafana queries `kpi\_vuln\_priority\_top50` and renders a sortable triage table showing the top 50 CVEs by `vuln\_priority\_score`, with KEV status, CVSS severity, and EPSS probability. The panel gives SOC analysts a continuously updated ranked list of vulnerabilities to prioritize for patching, combining exploit likelihood, active exploitation evidence, and technical severity into a single score.

### 4.2 IOC correlation feed

Grafana queries `exploitation/ioc\_correlations` and displays network events matched against known ThreatFox and URLhaus indicators. Each row shows the matched IOC, its associated malware family, and the source and destination of the network event. Analysts use this feed to identify hosts communicating with known malicious infrastructure without manually cross-referencing threat intelligence sources.

### 4.3 ML anomaly feed

The `cybersecintel\_ml\_ids\_anomaly` DAG task runs a two-stage Spark job. In the training stage, Spark reads `exploitation/network\_events`, extracts a feature matrix, and trains a scikit-learn Isolation Forest; the model is serialized to `s3://models/ids\_anomaly\_detector/`. In the inference stage, Spark loads the model, scores all flows, and writes anomalous records to `exploitation/anomaly\_flags`. Grafana queries this table and surfaces flows that deviate from baseline behavior, enabling analysts to detect novel threats that signature-based rules would miss.

### 4.4 Alert enrichment feed

A Spark Structured Streaming job implements the warm-path alert enrichment:

1. reads from Kafka `ids.alerts` using the `spark-sql-kafka` connector in micro-batch mode (10-second interval);
2. joins each batch against `exploitation/vuln\_enriched` loaded as a static broadcast DataFrame, attaching `vuln\_priority\_score` and `cvss\_v3\_severity`;
3. writes enriched records to the warm enriched Delta table in `s3://exploitation/`.

Grafana queries the warm enriched table and displays a live-updating view linking each IDS alert to its CVE context, giving analysts the evidence needed to assess alert severity within seconds of the alert arriving.

### 4.5 Live alert stream

The Kafka topic `suricata.events` surfaces raw Suricata EVE events in real time without further processing. Events are already normalized at publish time by the P1 replay pipeline. The live alert stream provides sub-second visibility into network activity and is consumed directly by analyst monitoring tools, bypassing the batch and micro-batch latency of the other endpoints.

## 5 Data Governance (Optional)

### 5.1 Data domains and products

The data in the exploitation zone belongs to four business domains:

| Domain | Data products |
|---|---|
| Vulnerability Intelligence | `exploitation/vuln\_enriched` (Delta), `exploitation/kpi/kpi\_vuln\_priority\_top` (Delta), `exploitation/kpi/kpi\_kev\_rolling7d` (Delta) |
| Network Security Events | `exploitation/network\_events` (Delta), `exploitation/kpi/kpi\_daily\_alert\_cour` (Delta), `exploitation/warm\_enriched` (Delta) |
| Threat Intelligence | `exploitation/ioc\_correlations` (Delta) |
| ML Artifacts | `s3://models/ids\_anomaly\_detector/` (MinIO, versioned by training date), `exploitation/anomaly\_flags` (Delta) |

### 5.2 Detailed data product specification: `exploitation/vuln_enriched`

| Attribute | Value |
|---|---|
| Name | `exploitation/vuln\_enriched` |
| Domain | Vulnerability Intelligence |
| Owner | CyberSecIntel data engineering team |
| Description | Pre-joined CVE enrichment table combining KEV exploit status, NVD CVSS severity, EPSS exploit probability, and CIRCL metadata into one row per CVE for analyst triage. |
| Update cadence | Daily (triggered by `cybersecintel\_exploitation\_zone` DAG). |
| Upstream lineage | `trusted/kev` $\leftarrow$ join $\rightarrow$ `trusted/nvd`, `trusted/epss`, `trusted/circl\_vulnlookup`. |
| Key | `cve\_id` (unique per row). |
| Quality checks | Non-null `cve\_id`; `epss\_score` $\in [0, 1]$; `cvss\_v3\_score` $\in [0, 10]$ or null; `vuln\_priority\_score` $\geq 0$. |
| Access policy | Readable by analysts, streaming job, and ML DAG task; write access restricted to the exploitation DAG service account. |
| Retention | Overwritten on each run; Delta Lake transaction log retained for 30 days to allow rollback. |

### 5.3 Lineage tracking

Each Spark job in the trusted and exploitation DAGs appends one row to a lineage Delta table at `s3://exploitation/lineage/` with columns: `run\_id`, `dag\_id`, `task\_id`, `source\_paths`, `target\_path`, `transformation\_name`, `rows\_read`, `rows\_written`, `rows\_rejected`, `started\_at`, `finished\_at`. Storing lineage in Delta keeps all pipeline metadata within the same storage layer, and the table is directly queryable by Grafana or Spark for pipeline health monitoring.

# 6 Constraint Coverage

| P2 constraint | Design response |
|---|---|
| C1 – Trusted Zone design | Storage (`s3://trusted/` + Delta), technology (Spark), and per-source cleaning transformations defined in Section 2. Three storage objects per zone: Delta clean tables, warm cleaned Delta, rejection tables. |
| C2 – Trusted Zone implementation | Spark jobs per source family and warm aggregates; Airflow DAG `cybersecintel\_trusted\_zone`; rejection side table at `trusted/rejected/`; lineage records in Delta. |
| C3 – Exploitation Zone design | Five Delta assets defined in Section 3: `vuln\_enriched`, `network\_events`, `ioc\_correlations`, `anomaly\_flags`, and KPI tables, each with join keys, schema, and computed columns. |
| C4 – Exploitation Zone implementation | Spark ETL with five idempotent tasks; Delta overwrite mode; KPI tables fully recomputed per run; DAG `cybersecintel\_exploitation\_zone`. |
| C5 – Consumption design and implementation | Five endpoints: CVE prioritization feed, IOC correlation feed, ML anomaly feed, and alert enrichment feed (all via Grafana querying Delta through Spark SQL), plus live alert stream (Kafka hot path, pass-through). Described in Section 4. |
| C6 – Architecture design | Updated diagram (Figure 1) shows all zones, storage objects, data paths (cold/warm/hot), and consumption endpoints. Section 1 explains the Option 1 rationale and per-tool justification. |
| C7 – Orchestration and containerization | Two new Airflow DAGs extend P1 orchestration. Grafana is the only new Docker service; MinIO, Spark, and Kafka are reused. |
| C8 – Organization | Code organized under `trusted/`, `exploitation/`, and `consumption/` top-level directories, each with per-source or per-task scripts. |

# 7 References


- Apache Spark documentation: https://spark.apache.org/docs/latest/
- Spark SQL Thrift Server: https://spark.apache.org/docs/latest/sql-distributed-sql-engine.html
- Delta Lake project: https://delta.io/
- MinIO object storage: https://min.io/
- Grafana documentation: https://grafana.com/docs/
- Apache Kafka documentation: https://kafka.apache.org/documentation/
- Apache Airflow documentation: https://airflow.apache.org/docs/
- scikit-learn Isolation Forest: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html
- P1 source references (KEV, NVD, EPSS, URLhaus, CIRCL, ThreatFox, CTU-13): see P1 Final Delivery, Section 2.3.